



Monte Carlo Simulation of Blackjack

Which way of playing loses the least over many hands?

Neeraj Srivastava | 25203038

ACM40960 Projects in Mathematics Modelling | MSc Data and Computational Science | University College Dublin
Supervisor: Dr Sarp Akcay | August 2026

Code, tests and data: github.com/ACM40960/projects-neeraj



What is this project?

Blackjack is a card game in which the player and dealer try to reach 21 without going over. This project asks a simple question: if a player follows one of three decision rules, how much should they expect to win or lose? A computer plays 100,000 rounds for each rule, records every outcome and calculates the average return. Formulae from the literature review check the mathematics; a fixed random seed, automated tests and validation experiments check that the result can be repeated and trusted.

The simple question

Which of three ways to play gives the smallest average loss—and how certain are we about that answer?

How the game and strategies work

Goal: finish closer to 21 than the dealer; a total above 21 is a bust and loses.
Game: one 52-card deck, dealer stands on every 17, and a natural Blackjack pays 3:2.
Naive: hit below 17 and stand on 17 or more.
Dealer-like: follow the dealer threshold but also hit a soft 17 (Ace + 6).
Basic: use the player's total, usable Ace and dealer's visible card to decide.

Formulae from the literature review

Formulae used in the simulation

$$(1) \quad \mathbb{P}(X_{k+1} = r | c_A, c_2, \dots, c_{10}) = \frac{N_r - c_r}{52 - k}$$
$$(2) \quad \mathbb{P}(\text{natural}) = 2 \binom{4}{52} \binom{16}{51} = \frac{128}{2652} \approx 0.0483$$

What one hand pays, Y_i :

- +1.5 player has a natural and dealer does not
- +1 player wins a normal hand
- 0 push
- −1 player loses

$$(3) \quad \mu_{\pi, R} = E[Y | \pi, R]$$

$$(4) \quad HE_{\pi, R} = -\mu_{\pi, R}$$

$$(5) \quad \hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n Y_i$$

$$(6) \quad \hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (Y_i - \hat{\mu}_n)^2, \quad SE(\hat{\mu}_n) = \frac{\hat{\sigma}}{\sqrt{n}}$$

$$(7) \quad 95\% \text{ CI} = \hat{\mu}_n \pm 1.96 \frac{\hat{\sigma}}{\sqrt{n}}$$

$$(8) \quad \Delta(\pi_1, \pi_2) = \hat{\mu}_{\pi_1, R} - \hat{\mu}_{\pi_2, R}$$

Equations (1)–(8) follow the uploaded literature review, Sections 3–4.

Quick guide — EV: average gain or loss per hand • House edge: casino advantage • SE: uncertainty in the estimate • 95% CI: likely range for the true EV

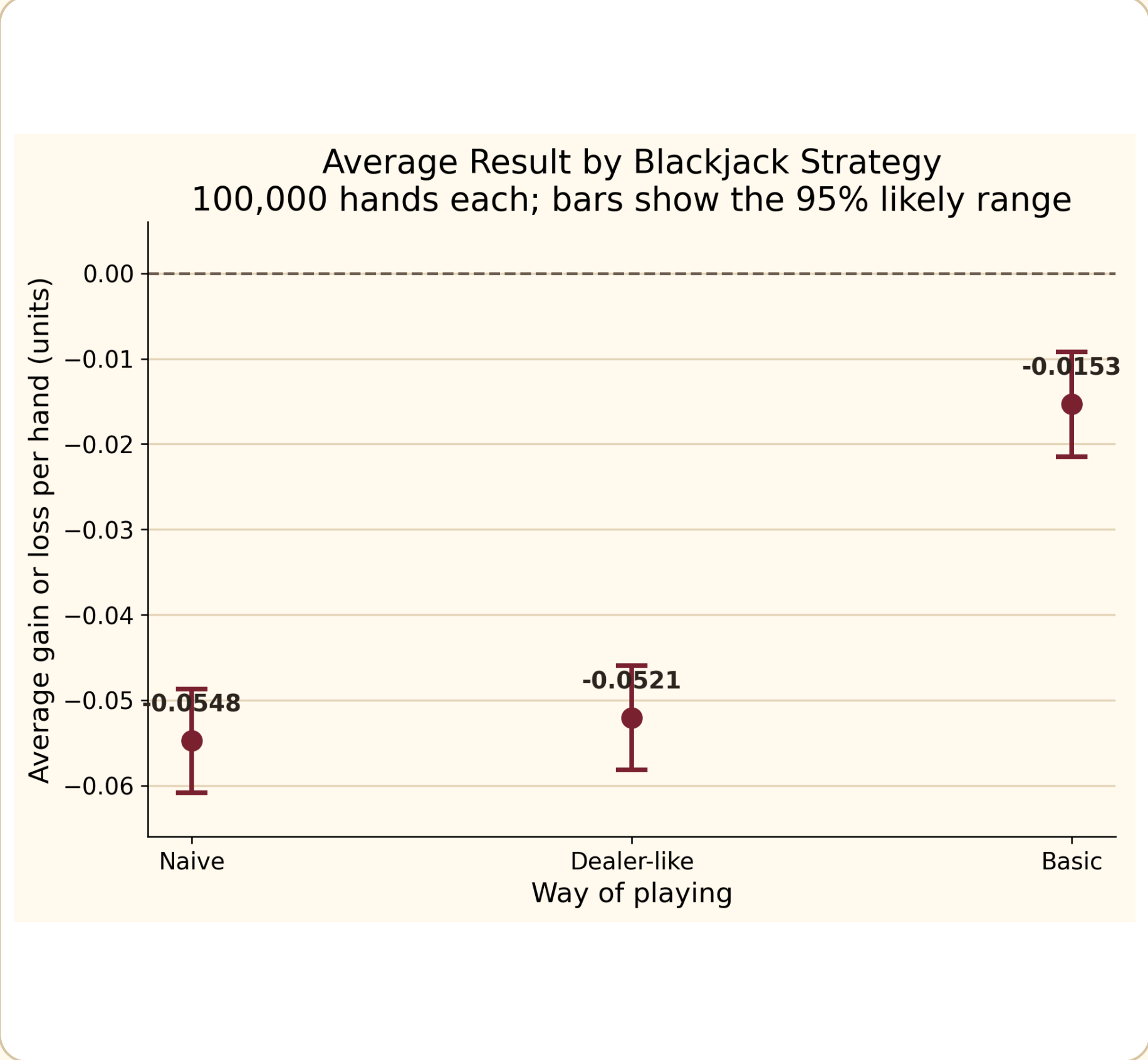
What the computer repeats

- Shuffle:** create a fresh deck using recorded seed 25203038.
- Deal:** give two cards to the player and dealer; check for naturals.
- Decide:** use one strategy to hit or stand until play ends.
- Score:** pay +1.5, +1, 0 or −1 unit according to the result.
- Repeat:** summarise 100,000 hands with rates, EV, SE and a 95% CI.

Why trust the result?

Every experiment records seed 25203038, so the same random sequence can be replayed. All three strategies use the same dealing, dealer and scoring code. The project has 98 passing automated tests, and the poster tables and figures come directly from the generated CSV results.

Figure 1. Average result per hand—closer to zero is better



Zero means break-even. A negative value means an expected loss. All three ranges are below zero, but basic strategy is much closer to break-even.

Per 100 one-unit hands, naive play loses about 5.48 units; basic strategy loses about 1.53. Basic play saves about 3.94 units.

Table 1A. What happened in the 100,000 hands?

Strategy	Rounds	Win %	Loss %	Push %	BJ %	Bust %
Naive	100,000	41.366	49.221	9.413	4.922	27.040
Dealer-like	100,000	41.553	49.139	9.308	4.922	27.424
Basic	100,000	43.840	47.752	8.408	4.922	16.913

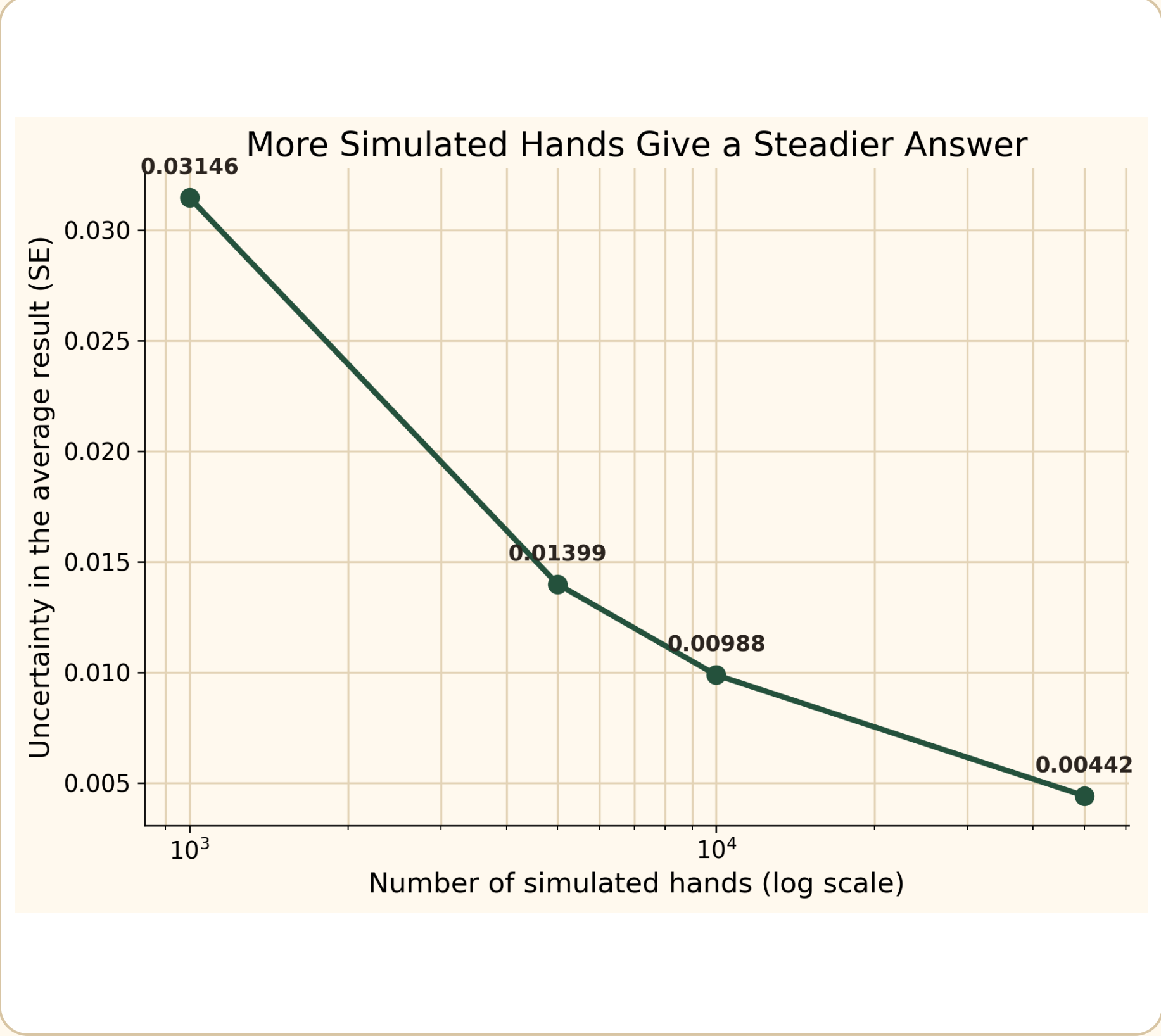
Table 1B. Average return and uncertainty (1 unit staked)

Strategy	EV	House edge %	SE	CI low	CI high
Naive	-0.054755	5.4755	0.003102	-0.060835	-0.048675
Dealer-like	-0.052065	5.2065	0.003104	-0.058150	-0.045980
Basic	-0.015325	1.5325	0.003123	-0.021446	-0.009204

What do the numbers say?

More information helps: basic play reacts to the hand type and the dealer's visible card.
Fewer busts: the player busts in 27.040% of naive hands but only 16.913% of basic hands.
More wins: basic play wins 2.474 percentage points more hands than naive play.
Smaller casino advantage: the house edge falls from 5.4755% to 1.5325%—about 72% lower.

Figure 2. More simulated hands give a steadier answer



Standard error falls from 0.03146 at 1,000 hands to 0.00442 at 50,000. In plain language, a larger simulation gives a less noisy estimate (Equation 6).

Table 2. The estimate becomes more precise

Rounds	EV	SE	CI low	CI high	CI width
1,000	+0.007500	0.031459	-0.054160	+0.069160	0.123320
5,000	-0.024500	0.013987	-0.051915	+0.002915	0.054831
10,000	-0.027150	0.009883	-0.046521	-0.007779	0.038741
50,000	-0.020620	0.004419	-0.029280	-0.011960	0.017321

Table 3. Does the simulator deal naturals correctly?

Rounds	Theory %	Observed %	SE pp	Diff pp	Tolerance pp	Result
100,000	4.8265	4.9220	0.0678	0.0955	0.2033	PASS

Theory predicts a natural on 4.8265% of opening hands; the simulation gives 4.9220%. The small difference passes the tolerance check, supporting Equation (2) and the deal code.

What the result means—and does not mean

Main result: choosing a strategy matters; basic play gives the best average result of the three.
Still a losing game: none of the simulated confidence intervals reaches break-even.
Simplified model: doubling, splitting, surrender, insurance and changing bets are not included.
Next step: compare deck counts, soft-17 rules and 3:2 versus 6:5 Blackjack payouts.

Bottom line

Basic strategy is the best of the three tested rules. It cuts the simulated casino advantage by about 72% compared with naive play, but it does not turn Blackjack into a profitable game. The formula, validation and convergence checks all support this conclusion.

References and project files

- [1] Baldwin et al. (1956), The Optimum Strategy in Blackjack.
 - [2] Griffin (1979), The Theory of Blackjack.
 - [3] Solowiej (2004), Finding Blackjack's optimal strategy in real-time and player's expected win.
 - [4] Sutton and Barto (2018), Reinforcement Learning: An Introduction.
- Results: seed 25203038. Repository: github.com/ACM40960/projects-neeraj



SCAN FOR CODE, TESTS & DATA

Python 3.12 | NumPy | pandas | Matplotlib | pytest